

Name : S.Dharanish

Reg no : 192472152

Couse code: CSA0925

Smart Campus Parking and Traffic Management System

1.PSEUDOCODE:

START

Connect to MySQL Database

IF connection fails

 Display "Database Connection Failed"

 STOP

END IF

Display Main Menu

REPEAT

 Display:

 1. User & Vehicle Management

 2. Parking Slot Management

 3. Reservation

 4. Entry / Exit

 5. Payments

 6. Violations

 7. Reports

 8. Exit

 Read choice

 IF choice = 1

 Add, search, update or display users

 Add and manage vehicle details

 Store data in database

 ELSE IF choice = 2

 Display parking slots

 Check slot status

 Update slot as Available, Occupied or Reserved

ELSE IF choice = 3

Read User ID, Vehicle ID and Slot ID

Check whether slot is Available

Create reservation

Change slot status to Reserved

ELSE IF choice = 4

Read Vehicle ID

Record vehicle entry

Create parking session

Record exit time

Calculate parking duration and amount

ELSE IF choice = 5

Read Session ID and Amount

Select Cash, UPI or Card

Store payment details

Set payment status

Display payment confirmation

ELSE IF choice = 6

Read User ID, Vehicle ID and Violation Type

Record description and fine amount

Store violation in database

Display violation confirmation

ELSE IF choice = 7

Calculate total users

Calculate total vehicles

Calculate available, occupied and reserved slots

Calculate reservations, payments and violations

Calculate total revenue and fines

Display reports

ELSE IF choice = 8

Close database connection

Display "Thank You"

EXIT

```
ELSE
    Display "Invalid Choice"
END IF
UNTIL choice = 8
STOP
```

2. Implementation & Results

2.1 Front-End Design:

```
import javax.swing.*;
import java.awt.*;

public class Main extends JFrame {

    public Main() {

        setTitle("Smart Campus Parking and Traffic Management System");
        setSize(1000, 650);
        setLocationRelativeTo(null);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        JLabel title = new JLabel(
            "SMART CAMPUS PARKING & TRAFFIC MANAGEMENT SYSTEM",
            SwingConstants.CENTER
        );

        title.setFont(new Font("Arial", Font.BOLD, 25));

        title.setBorder(
            BorderFactory.createEmptyBorder(
                25, 10, 25, 10
            )
        );

        JPanel panel = new JPanel(
            new GridLayout(4, 2, 20, 20)
        );

        panel.setBorder(
            BorderFactory.createEmptyBorder(
                30, 60, 40, 60
            )
        );

        JButton userVehicleButton =
            new JButton("User & Vehicle");

        JButton parkingButton =
```

```

        new JButton("Parking Slots");

JButton reservationButton =
    new JButton("Reservation");

JButton entryExitButton =
    new JButton("Entry / Exit");

JButton paymentButton =
    new JButton("Payments");

JButton violationButton =
    new JButton("Violations");

JButton reportsButton =
    new JButton("Reports");

JButton exitButton =
    new JButton("Exit");

Font buttonFont =
    new Font("Arial", Font.BOLD, 16);

userVehicleButton.setFont(buttonFont);
parkingButton.setFont(buttonFont);
reservationButton.setFont(buttonFont);
entryExitButton.setFont(buttonFont);
paymentButton.setFont(buttonFont);
violationButton.setFont(buttonFont);
reportsButton.setFont(buttonFont);
exitButton.setFont(buttonFont);

panel.add(userVehicleButton);
panel.add(parkingButton);

panel.add(reservationButton);
panel.add(entryExitButton);

panel.add(paymentButton);
panel.add(violationButton);

panel.add(reportsButton);
panel.add(exitButton);

add(title, BorderLayout.NORTH);
add(panel, BorderLayout.CENTER);

userVehicleButton.addActionListener(e -> {
    new UserVehicle();
});

parkingButton.addActionListener(e -> {
    new ParkingSlots();
});

reservationButton.addActionListener(e -> {

```

```

        new Reservation();
    });

    entryExitButton.addActionListener(e -> {
        new EntryExit();
    });

    paymentButton.addActionListener(e -> {
        new Payments();
    });

    violationButton.addActionListener(e -> {
        new Violations();
    });

    reportsButton.addActionListener(e -> {
        new Reports();
    });

    exitButton.addActionListener(e -> {
        System.exit(0);
    });

    setVisible(true);
}

public static void main(String[] args) {

    SwingUtilities.invokeLater(() -> {
        new Main();
    });
}
}

```

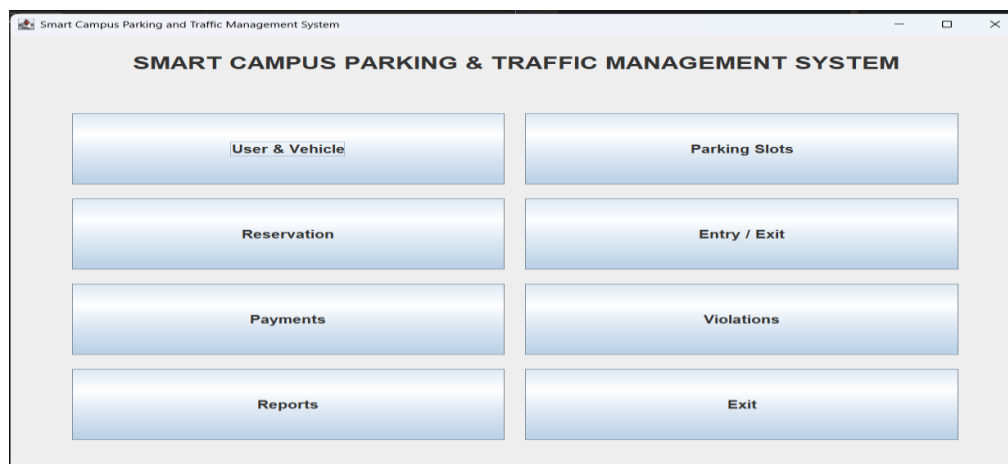


Figure 1: Main Dashboard of the Smart Campus Parking and Traffic Management System

2.2 Back-End Design:

```
CREATE DATABASE IF NOT EXISTS smart_campus_parking;
```

```
USE smart_campus_parking;
```

```
CREATE TABLE users (  
    user_id INT PRIMARY KEY AUTO_INCREMENT,  
    name VARCHAR(100) NOT NULL,  
    email VARCHAR(100) NOT NULL UNIQUE,  
    phone VARCHAR(15) NOT NULL,  
    user_type ENUM('Student','Faculty','Staff','Visitor') NOT NULL  
);
```

```
CREATE TABLE vehicles (  
    vehicle_id INT PRIMARY KEY AUTO_INCREMENT,  
    user_id INT NOT NULL,  
    vehicle_number VARCHAR(20) NOT NULL UNIQUE,  
    vehicle_type VARCHAR(30) NOT NULL,  
    FOREIGN KEY (user_id) REFERENCES users(user_id)  
);
```

```
CREATE TABLE parking_zones (  
    zone_id INT PRIMARY KEY AUTO_INCREMENT,  
    zone_name VARCHAR(50) NOT NULL,  
    location VARCHAR(100)  
);
```

```
CREATE TABLE parking_slots (  
    slot_id INT PRIMARY KEY AUTO_INCREMENT,  
    zone_id INT NOT NULL,  
    slot_number VARCHAR(20) NOT NULL,  
    slot_status ENUM('Available','Occupied','Reserved') DEFAULT 'Available',  
    FOREIGN KEY (zone_id) REFERENCES parking_zones(zone_id)  
);
```

```
CREATE TABLE reservations (  
    reservation_id INT PRIMARY KEY AUTO_INCREMENT,  
    user_id INT NOT NULL,  
    vehicle_id INT NOT NULL,  
    slot_id INT NOT NULL,  
    reservation_date DATETIME DEFAULT CURRENT_TIMESTAMP,  
    status ENUM('Active','Completed','Cancelled') DEFAULT 'Active',  
    FOREIGN KEY (user_id) REFERENCES users(user_id),  
    FOREIGN KEY (vehicle_id) REFERENCES vehicles(vehicle_id),  
    FOREIGN KEY (slot_id) REFERENCES parking_slots(slot_id)  
);
```

```
CREATE TABLE parking_sessions (  
    session_id INT PRIMARY KEY AUTO_INCREMENT,  
    vehicle_id INT NOT NULL,  
    slot_id INT NOT NULL,  
    entry_time DATETIME DEFAULT CURRENT_TIMESTAMP,  
    exit_time DATETIME NULL,  
    duration_minutes INT DEFAULT 0,  
    parking_amount DECIMAL(10,2) DEFAULT 0.00,  
    session_status ENUM('Active','Completed') DEFAULT 'Active',  
    FOREIGN KEY (vehicle_id) REFERENCES vehicles(vehicle_id),  
    FOREIGN KEY (slot_id) REFERENCES parking_slots(slot_id)  
);
```

```
CREATE TABLE payments (  
    payment_id INT PRIMARY KEY AUTO_INCREMENT,  
    session_id INT NOT NULL,  
    amount DECIMAL(10,2) NOT NULL,  
    payment_method ENUM('Cash','UPI','Card') NOT NULL,  
    payment_date DATETIME DEFAULT CURRENT_TIMESTAMP,  
    payment_status ENUM('Paid','Pending') DEFAULT 'Paid',  
    FOREIGN KEY (session_id) REFERENCES parking_sessions(session_id)
```

);

```
CREATE TABLE violations (  
    violation_id INT PRIMARY KEY AUTO_INCREMENT,  
    user_id INT,  
    vehicle_id INT NOT NULL,  
    violation_type VARCHAR(100) NOT NULL,  
    description VARCHAR(255),  
    fine_amount DECIMAL(10,2) DEFAULT 0.00,  
    violation_date DATETIME DEFAULT CURRENT_TIMESTAMP,  
    status VARCHAR(20) NOT NULL,  
    FOREIGN KEY (user_id) REFERENCES users(user_id),  
    FOREIGN KEY (vehicle_id) REFERENCES vehicles(vehicle_id)  
);
```

```
mysql> SHOW TABLES;  
+-----+  
| Tables_in_smart_campus_parking |  
+-----+  
| parking_passes  
| parking_sessions  
| parking_slots  
| parking_zones  
| payments  
| reservations  
| users  
| vehicles  
| violations  
+-----+  
9 rows in set (0.07 sec)  
  
mysql> |
```

Figure 2: MySQL Database Tables

```
mysql> DESCRIBE users;  
+-----+-----+-----+-----+-----+-----+  
| Field | Type | Null | Key | Default | Extra |  
+-----+-----+-----+-----+-----+-----+  
| user_id | int | NO | PRI | NULL | auto_increment |  
| name | varchar(100) | NO | | NULL | |  
| email | varchar(100) | NO | UNI | NULL | |  
| phone | varchar(15) | NO | | NULL | |  
| user_type | enum('Student','Faculty','Staff','Visitor','Service') | NO | | NULL | |  
+-----+-----+-----+-----+-----+-----+  
5 rows in set (0.02 sec)
```



```
mysql> DESCRIBE vehicles;
```

Field	Type	Null	Key	Default	Extra
vehicle_id	int	NO	PRI	NULL	auto_increment
user_id	int	NO	MUL	NULL	
vehicle_number	varchar(20)	NO	UNI	NULL	
vehicle_type	enum('Car','Bike','Van','Bus','Other')	NO		NULL	

4 rows in set (0.00 sec)

```
mysql> DESCRIBE parking_slots;
```

Field	Type	Null	Key	Default	Extra
slot_id	int	NO	PRI	NULL	auto_increment
zone_id	int	NO	MUL	NULL	
slot_number	varchar(20)	NO		NULL	
slot_status	enum('Available','Occupied','Reserved')	YES		Available	

4 rows in set (0.01 sec)

```
mysql> DESCRIBE reservations;
```

Field	Type	Null	Key	Default	Extra
reservation_id	int	NO	PRI	NULL	auto_increment
user_id	int	NO	MUL	NULL	
vehicle_id	int	NO	MUL	NULL	
slot_id	int	NO	MUL	NULL	
reservation_date	date	NO		NULL	
start_time	time	NO		NULL	
end_time	time	NO		NULL	
status	enum('Reserved','Cancelled','Completed')	YES		Reserved	

8 rows in set (0.00 sec)

```
mysql> DESCRIBE parking_sessions;
```

Field	Type	Null	Key	Default	Extra
session_id	int	NO	PRI	NULL	auto_increment
vehicle_id	int	NO	MUL	NULL	
slot_id	int	NO	MUL	NULL	
entry_time	datetime	NO		NULL	
exit_time	datetime	YES		NULL	
duration_minutes	int	YES		NULL	
parking_fee	decimal(10,2)	YES		0.00	
status	enum('Active','Completed')	YES		Active	

8 rows in set (0.00 sec)

```
mysql> DESCRIBE payments;
```

Field	Type	Null	Key	Default	Extra
payment_id	int	NO	PRI	NULL	auto_increment
session_id	int	NO	MUL	NULL	
amount	decimal(10,2)	NO		NULL	
payment_method	enum('Cash','UPI','Card')	NO		NULL	
payment_date	datetime	YES		CURRENT_TIMESTAMP	DEFAULT_GENERATED
payment_status	enum('Paid','Pending')	YES		Paid	

6 rows in set (0.00 sec)

```
mysql> DESCRIBE violations;
```

Field	Type	Null	Key	Default	Extra
violation_id	int	NO	PRI	NULL	auto_increment
user_id	int	YES	MUL	NULL	
vehicle_id	int	NO	MUL	NULL	
violation_type	varchar(100)	NO		NULL	
description	varchar(255)	YES		NULL	
fine_amount	decimal(10,2)	YES		0.00	
violation_date	datetime	YES		CURRENT_TIMESTAMP	DEFAULT_GENERATED
status	varchar(20)	NO		NULL	

```
8 rows in set (0.00 sec)
```

2.3 Implementation:

User & Vehicle Management:

```
import javax.swing.*;
import java.awt.*;
import java.sql.*;

public class UserVehicle extends JFrame {

    JTextField nameField, emailField, phoneField;
    JTextField vehicleNumberField, vehicleUserIdField;
    JComboBox<String> userTypeBox, vehicleTypeBox;
    JTextArea outputArea;

    public UserVehicle() {

        setTitle("User & Vehicle Management");
        setSize(850, 650);
        setLocationRelativeTo(null);
        setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);

        JLabel title = new JLabel("USER & VEHICLE MANAGEMENT",
        SwingConstants.CENTER);
        title.setFont(new Font("Arial", Font.BOLD, 22));
        title.setBorder(BorderFactory.createEmptyBorder(15, 10, 15, 10));

        JPanel userPanel = new JPanel(new GridLayout(5, 2, 10, 10));
        userPanel.setBorder(BorderFactory.createTitledBorder("User Details"));

        nameField = new JTextField();
        emailField = new JTextField();
        phoneField = new JTextField();

        userTypeBox = new JComboBox<>(
            new String[]{"Student", "Faculty", "Staff", "Visitor", "Service"}
        );
    }
}
```

```
JButton addUserButton = new JButton("Add User");
JButton viewUserButton = new JButton("View Users");

userPanel.add(new JLabel("Name:"));
userPanel.add(nameField);

userPanel.add(new JLabel("Email:"));
userPanel.add(emailField);

userPanel.add(new JLabel("Phone:"));
userPanel.add(phoneField);

userPanel.add(new JLabel("User Type:"));
userPanel.add(userTypeBox);

userPanel.add(addUserButton);
userPanel.add(viewUserButton);

JPanel vehiclePanel = new JPanel(new GridLayout(4, 2, 10, 10));
vehiclePanel.setBorder(BorderFactory.createTitledBorder("Vehicle Details"));

vehicleUserIdField = new JTextField();
vehicleNumberField = new JTextField();

vehicleTypeBox = new JComboBox<>(
    new String[] {"Car", "Bike", "Van", "Bus", "Other"}
);

JButton addVehicleButton = new JButton("Add Vehicle");
JButton viewVehicleButton = new JButton("View Vehicles");

vehiclePanel.add(new JLabel("User ID:"));
vehiclePanel.add(vehicleUserIdField);

vehiclePanel.add(new JLabel("Vehicle Number:"));
vehiclePanel.add(vehicleNumberField);

vehiclePanel.add(new JLabel("Vehicle Type:"));
vehiclePanel.add(vehicleTypeBox);

vehiclePanel.add(addVehicleButton);
vehiclePanel.add(viewVehicleButton);

outputArea = new JTextArea();
outputArea.setEditable(false);
```

```

outputArea.setFont(new Font("Monospaced", Font.PLAIN, 14));

JScrollPane scrollPane = new JScrollPane(outputArea);
scrollPane.setBorder(BorderFactory.createTitledBorder("Database Records"));

JPanel centerPanel = new JPanel(new GridLayout(2, 1, 10, 10));
centerPanel.setBorder(BorderFactory.createEmptyBorder(10, 20, 10, 20));

centerPanel.add(userPanel);
centerPanel.add(vehiclePanel);

add(title, BorderLayout.NORTH);
add(centerPanel, BorderLayout.CENTER);
add(scrollPane, BorderLayout.SOUTH);

addUserButton.addActionListener(e -> addUser());
viewUserButton.addActionListener(e -> viewUsers());
addVehicleButton.addActionListener(e -> addVehicle());
viewVehicleButton.addActionListener(e -> viewVehicles());

setVisible(true);
}

private void addUser() {

    String name = nameField.getText().trim();
    String email = emailField.getText().trim();
    String phone = phoneField.getText().trim();
    String type = userTypeBox.getSelectedItem().toString();

    if (name.isEmpty() || email.isEmpty() || phone.isEmpty()) {
        JOptionPane.showMessageDialog(this, "Please fill all user details.");
        return;
    }

    String sql = "INSERT INTO users (name, email, phone, user_type) VALUES (?, ?, ?,
?)"

    try (Connection con = DBConnection.getConnection();
        PreparedStatement ps = con.prepareStatement(sql)) {

        ps.setString(1, name);
        ps.setString(2, email);
        ps.setString(3, phone);
        ps.setString(4, type);
    }
}

```

```

ps.executeUpdate();

JOptionPane.showMessageDialog(this, "User added successfully.");

nameField.setText("");
emailField.setText("");
phoneField.setText("");

} catch (SQLException e) {
    JOptionPane.showMessageDialog(this, "Error: " + e.getMessage());
}
}

private void viewUsers() {

    String sql = "SELECT * FROM users";

    try (Connection con = DBConnection.getConnection();
        Statement st = con.createStatement();
        ResultSet rs = st.executeQuery(sql)) {

        outputArea.setText("");

        while (rs.next()) {

            outputArea.append(
                "ID: " + rs.getInt("user_id") +
                " | Name: " + rs.getString("name") +
                " | Email: " + rs.getString("email") +
                " | Phone: " + rs.getString("phone") +
                " | Type: " + rs.getString("user_type") +
                "\n"
            );

        }

    } catch (SQLException e) {
        JOptionPane.showMessageDialog(this, "Error: " + e.getMessage());
    }
}

private void addVehicle() {

    String userIdText = vehicleUserIdField.getText().trim();
    String vehicleNumber = vehicleNumberField.getText().trim();
    String vehicleType = vehicleTypeBox.getSelectedItem().toString();

```

```

if (userIdText.isEmpty() || vehicleNumber.isEmpty()) {
    JOptionPane.showMessageDialog(this, "Please fill all vehicle details.");
    return;
}

try {

    int userId = Integer.parseInt(userIdText);

    String sql = "INSERT INTO vehicles (user_id, vehicle_number, vehicle_type)
VALUES (?, ?, ?)";

    try (Connection con = DBConnection.getConnection();
        PreparedStatement ps = con.prepareStatement(sql)) {

        ps.setInt(1, userId);
        ps.setString(2, vehicleNumber);
        ps.setString(3, vehicleType);

        ps.executeUpdate();

        JOptionPane.showMessageDialog(this, "Vehicle added successfully.");

        vehicleUserIdField.setText("");
        vehicleNumberField.setText("");
    }

} catch (NumberFormatException e) {

    JOptionPane.showMessageDialog(this, "User ID must be a number.");

} catch (SQLException e) {

    JOptionPane.showMessageDialog(this, "Error: " + e.getMessage());
}

}

private void viewVehicles() {

    String sql = "SELECT * FROM vehicles";

    try (Connection con = DBConnection.getConnection();
        Statement st = con.createStatement();
        ResultSet rs = st.executeQuery(sql)) {

        outputArea.setText("");
    }
}

```

```

while (rs.next()) {

    outputArea.append(
        "Vehicle ID: " + rs.getInt("vehicle_id") +
        " | User ID: " + rs.getInt("user_id") +
        " | Number: " + rs.getString("vehicle_number") +
        " | Type: " + rs.getString("vehicle_type") +
        "\n"
    );
}

} catch (SQLException e) {
    JOptionPane.showMessageDialog(this, "Error: " + e.getMessage());
}
}
}

```

The screenshot shows a Java Swing application window titled "User & Vehicle Management". The window contains two main panels. The top panel, titled "User Details", has four input fields: "Name:", "Email:", "Phone:", and "User Type:". The "User Type:" field is a dropdown menu currently showing "Student". Below these fields are two buttons: "Add User" and "View Users". The bottom panel, titled "Vehicle Details", has three input fields: "User ID:", "Vehicle Number:", and "Vehicle Type:". The "Vehicle Type:" field is a dropdown menu currently showing "Car". Below these fields are two buttons: "Add Vehicle" and "View Vehicles". At the very bottom of the window is a section labeled "Database Records" which is currently empty.

Figure 4: User & Vehicle Management

Parking Slot Management:

```
import javax.swing.*;
import java.awt.*;
import java.sql.*;

public class ParkingSlots extends JFrame {

    JTextArea outputArea;

    public ParkingSlots() {

        setTitle("Parking Slot Management");
        setSize(800, 600);
        setLocationRelativeTo(null);
        setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);

        JLabel title = new JLabel(
            "PARKING SLOT MANAGEMENT",
            SwingConstants.CENTER
        );

        title.setFont(new Font("Arial", Font.BOLD, 24));
        title.setBorder(
            BorderFactory.createEmptyBorder(20, 10, 20, 10)
        );

        JPanel buttonPanel = new JPanel(
            new GridLayout(1, 3, 15, 15)
        );

        JButton viewButton = new JButton("View All Slots");
        JButton availableButton = new JButton("Available Slots");
        JButton refreshButton = new JButton("Refresh");

        buttonPanel.add(viewButton);
        buttonPanel.add(availableButton);
        buttonPanel.add(refreshButton);

        outputArea = new JTextArea();
        outputArea.setEditable(false);
        outputArea.setFont(
            new Font("Monospaced", Font.PLAIN, 15)
        );
    }
}
```



```

JScrollPane scrollPane = new JScrollPane(outputArea);

JPanel mainPanel = new JPanel(new BorderLayout(10, 10));
mainPanel.setBorder(
    BorderFactory.createEmptyBorder(20, 30, 20, 30)
);

mainPanel.add(buttonPanel, BorderLayout.NORTH);
mainPanel.add(scrollPane, BorderLayout.CENTER);

add(title, BorderLayout.NORTH);
add(mainPanel, BorderLayout.CENTER);

viewButton.addActionListener(e -> viewAllSlots());

availableButton.addActionListener(e -> viewAvailableSlots());

refreshButton.addActionListener(e -> viewAllSlots());

setVisible(true);

viewAllSlots();
}

private void viewAllSlots() {

    String sql =
        "SELECT ps.slot_id, pz.zone_name, " +
        "ps.slot_number, ps.slot_status " +
        "FROM parking_slots ps " +
        "JOIN parking_zones pz " +
        "ON ps.zone_id = pz.zone_id " +
        "ORDER BY ps.slot_id";

    try (
        Connection con = DBConnection.getConnection();
        Statement st = con.createStatement();
        ResultSet rs = st.executeQuery(sql)
    ) {

        outputArea.setText("");

        outputArea.append(
            String.format(
                "%-8s %-20s %-15s %-15s%n",
                "ID",

```

```

        "ZONE",
        "SLOT",
        "STATUS"
    )
);

outputArea.append(
    "-----\n"
);

while (rs.next()) {

    outputArea.append(
        String.format(
            "%-8d %-20s %-15s %-15s%n",
            rs.getInt("slot_id"),
            rs.getString("zone_name"),
            rs.getString("slot_number"),
            rs.getString("slot_status")
        )
    );
}

} catch (SQLException e) {

    JOptionPane.showMessageDialog(
        this,
        "Database Error: " + e.getMessage()
    );
}

}

private void viewAvailableSlots() {

    String sql =
        "SELECT ps.slot_id, pz.zone_name, " +
        "ps.slot_number, ps.slot_status " +
        "FROM parking_slots ps " +
        "JOIN parking_zones pz " +
        "ON ps.zone_id = pz.zone_id " +
        "WHERE ps.slot_status = 'Available' " +
        "ORDER BY ps.slot_id";

    try (
        Connection con = DBConnection.getConnection();
        Statement st = con.createStatement();
    ) {

```

```

        ResultSet rs = st.executeQuery(sql)
    ) {

        outputArea.setText("");

        outputArea.append(
            String.format(
                "%-8s %-20s %-15s%n",
                "ID",
                "ZONE",
                "SLOT"
            )
        );

        outputArea.append(
            "-----\n"
        );

        while (rs.next()) {

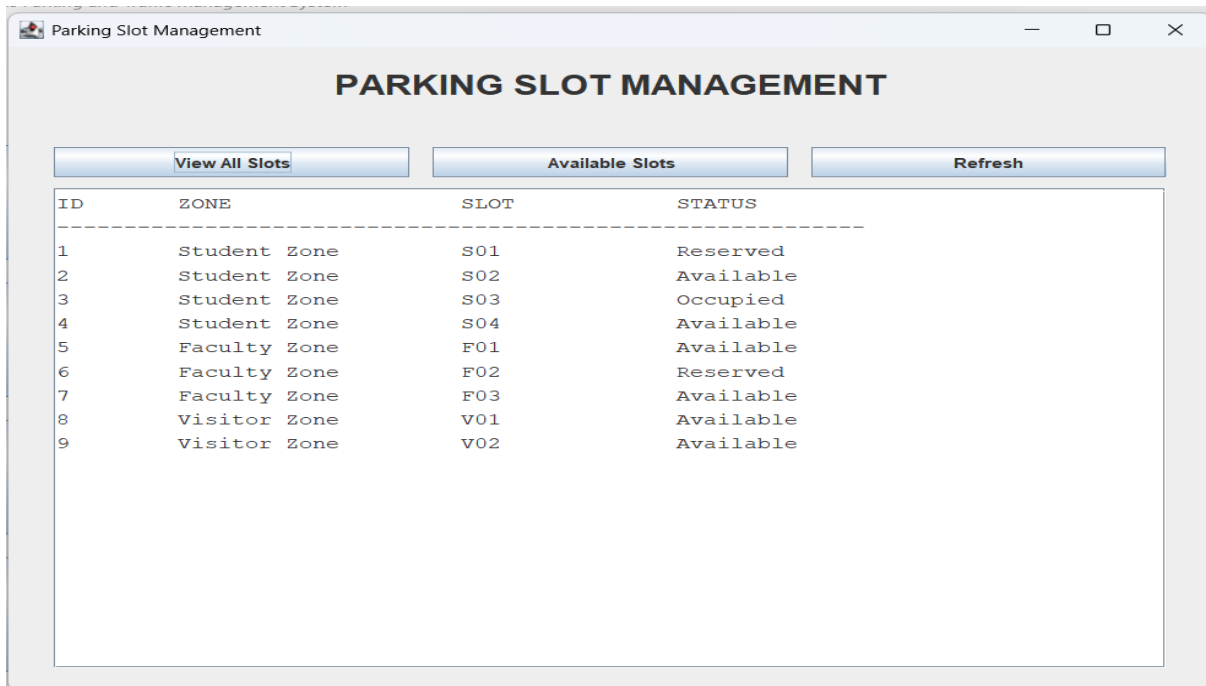
            outputArea.append(
                String.format(
                    "%-8d %-20s %-15s%n",
                    rs.getInt("slot_id"),
                    rs.getString("zone_name"),
                    rs.getString("slot_number")
                )
            );
        }

    } catch (SQLException e) {

        JOptionPane.showMessageDialog(
            this,
            "Database Error: " + e.getMessage()
        );
    }
}

```

Figure 5: Parking Slot Management



ID	ZONE	SLOT	STATUS
1	Student Zone	S01	Reserved
2	Student Zone	S02	Available
3	Student Zone	S03	Occupied
4	Student Zone	S04	Available
5	Faculty Zone	F01	Available
6	Faculty Zone	F02	Reserved
7	Faculty Zone	F03	Available
8	Visitor Zone	V01	Available
9	Visitor Zone	V02	Available

Reservation Management

```
import javax.swing.*;
import java.awt.*;
import java.sql.*;
import java.time.LocalDate;
import java.time.LocalDateTime;
import java.time.format.DateTimeParseException;
```

```
public class Reservation extends JFrame {
```

```
    JTextField userIdField;
    JTextField vehicleIdField;
    JTextField dateField;
    JTextField startTimeField;
    JTextField endTimeField;
```

```
    JComboBox<String> slotBox;
```

```
    JTextArea outputArea;
```

```
    public Reservation() {
```

```

setTitle("Parking Reservation Management");
setSize(850, 650);
setLocationRelativeTo(null);
setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);

JLabel title = new JLabel(
    "PARKING RESERVATION MANAGEMENT",
    SwingConstants.CENTER
);

title.setFont(new Font("Arial", Font.BOLD, 24));
title.setBorder(
    BorderFactory.createEmptyBorder(20, 10, 20, 10)
);

JPanel formPanel = new JPanel(new GridLayout(6, 2, 10, 10));

formPanel.setBorder(
    BorderFactory.createTitledBorder("Reservation Details")
);

userIdField = new JTextField();
vehicleIdField = new JTextField();
dateField = new JTextField(
    LocalDate.now().toString()
);
startTimeField = new JTextField("09:00");
endTimeField = new JTextField("10:00");

slotBox = new JComboBox<>();

formPanel.add(new JLabel("User ID:"));
formPanel.add(userIdField);

formPanel.add(new JLabel("Vehicle ID:"));
formPanel.add(vehicleIdField);

formPanel.add(new JLabel("Parking Slot:"));
formPanel.add(slotBox);

formPanel.add(new JLabel("Date (YYYY-MM-DD):"));
formPanel.add(dateField);

formPanel.add(new JLabel("Start Time (HH:MM):"));
formPanel.add(startTimeField);

```

```

formPanel.add(new JLabel("End Time (HH:MM):"));
formPanel.add(endTimeField);

JPanel buttonPanel = new JPanel(
    new GridLayout(1, 3, 15, 15)
);

JButton reserveButton = new JButton("Reserve Slot");
JButton viewButton = new JButton("View Reservations");
JButton refreshButton = new JButton("Refresh Slots");

buttonPanel.add(reserveButton);
buttonPanel.add(viewButton);
buttonPanel.add(refreshButton);

outputArea = new JTextArea();
outputArea.setEditable(false);
outputArea.setFont(
    new Font("Monospaced", Font.PLAIN, 14)
);

JScrollPane scrollPane = new JScrollPane(outputArea);

JPanel centerPanel = new JPanel(new BorderLayout(10, 10));

centerPanel.setBorder(
    BorderFactory.createEmptyBorder(15, 25, 15, 25)
);

centerPanel.add(formPanel, BorderLayout.NORTH);
centerPanel.add(buttonPanel, BorderLayout.CENTER);
centerPanel.add(scrollPane, BorderLayout.SOUTH);

add(title, BorderLayout.NORTH);
add(centerPanel, BorderLayout.CENTER);

reserveButton.addActionListener(e -> reserveSlot());

viewButton.addActionListener(e -> viewReservations());

refreshButton.addActionListener(e -> loadAvailableSlots());

loadAvailableSlots();

setVisible(true);
}

```

```

private void loadAvailableSlots() {

    slotBox.removeAllItems();

    String sql =
        "SELECT slot_id, slot_number " +
        "FROM parking_slots " +
        "WHERE slot_status = 'Available' " +
        "ORDER BY slot_id";

    try (
        Connection con = DBConnection.getConnection();
        Statement st = con.createStatement();
        ResultSet rs = st.executeQuery(sql)
    ) {

        while (rs.next()) {

            String item =
                rs.getInt("slot_id") +
                " - " +
                rs.getString("slot_number");

            slotBox.addItem(item);
        }

    } catch (SQLException e) {

        JOptionPane.showMessageDialog(
            this,
            "Database Error: " + e.getMessage()
        );
    }
}

private void reserveSlot() {

    String userText = userIdField.getText().trim();
    String vehicleText = vehicleIdField.getText().trim();
    String dateText = dateField.getText().trim();
    String startText = startTimeField.getText().trim();
    String endText = endTimeField.getText().trim();

    if (userText.isEmpty() ||
        vehicleText.isEmpty() ||

```

```

        dateText.isEmpty() ||
        startText.isEmpty() ||
        endText.isEmpty()) {

    JOptionPane.showMessageDialog(
        this,
        "Please fill all reservation details."
    );

    return;
}

if (slotBox.getSelectedItem() == null) {

    JOptionPane.showMessageDialog(
        this,
        "No available parking slots."
    );

    return;
}

try {

    int userId = Integer.parseInt(userText);
    int vehicleId = Integer.parseInt(vehicleText);

    LocalDate reservationDate =
        LocalDate.parse(dateText);

    LocalTime startTime =
        LocalTime.parse(startText);

    LocalTime endTime =
        LocalTime.parse(endText);

    if (!endTime.isAfter(startTime)) {

        JOptionPane.showMessageDialog(
            this,
            "End time must be after start time."
        );

        return;
    }
}

```



```

String selectedSlot =
    slotBox.getSelectedItem().toString();

int slotId =
    Integer.parseInt(
        selectedSlot.split(" - ")[0]
    );

Connection con = null;

try {

    con = DBConnection.getConnection();

    con.setAutoCommit(false);

    String vehicleCheck =
        "SELECT vehicle_id FROM vehicles " +
        "WHERE vehicle_id = ? AND user_id = ?";

    try (PreparedStatement ps =
        con.prepareStatement(vehicleCheck)) {

        ps.setInt(1, vehicleId);
        ps.setInt(2, userId);

        ResultSet rs = ps.executeQuery();

        if (!rs.next()) {

            con.rollback();

            JOptionPane.showMessageDialog(
                this,
                "Vehicle does not belong to this user."
            );

            return;
        }
    }

    String slotCheck =
        "SELECT slot_status FROM parking_slots " +
        "WHERE slot_id = ? FOR UPDATE";

    try (PreparedStatement ps =

```

```

        con.prepareStatement(slotCheck)) {

    ps.setInt(1, slotId);

    ResultSet rs = ps.executeQuery();

    if (!rs.next()) {

        con.rollback();

        JOptionPane.showMessageDialog(
            this,
            "Parking slot not found."
        );

        return;
    }

    String status =
        rs.getString("slot_status");

    if (!status.equals("Available")) {

        con.rollback();

        JOptionPane.showMessageDialog(
            this,
            "This parking slot is not available."
        );

        loadAvailableSlots();

        return;
    }
}

String insertSql =
    "INSERT INTO reservations " +
        "(user_id, vehicle_id, slot_id, reservation_date, start_time, end_time,
status) " +
        "VALUES (?, ?, ?, ?, ?, ?, 'Reserved)";

try (PreparedStatement ps =
    con.prepareStatement(insertSql)) {

    ps.setInt(1, userId);

```

```

        ps.setInt(2, vehicleId);
        ps.setInt(3, slotId);
        ps.setDate(
            4,
            Date.valueOf(reservationDate)
        );
        ps.setTime(
            5,
            Time.valueOf(startTime)
        );
        ps.setTime(
            6,
            Time.valueOf(endTime)
        );

        ps.executeUpdate();
    }

    String updateSql =
        "UPDATE parking_slots " +
        "SET slot_status = 'Reserved' " +
        "WHERE slot_id = ?";

    try (PreparedStatement ps =
        con.prepareStatement(updateSql)) {

        ps.setInt(1, slotId);

        ps.executeUpdate();
    }

    con.commit();

    JOptionPane.showMessageDialog(
        this,
        "Parking slot reserved successfully!"
    );

    userIdField.setText("");
    vehicleIdField.setText("");
    startTimeField.setText("09:00");
    endTimeField.setText("10:00");

    loadAvailableSlots();

} catch (SQLException e) {

```

```

        if (con != null) {
            try {
                con.rollback();
            } catch (SQLException ignored) {
            }
        }

        JOptionPane.showMessageDialog(
            this,
            "Reservation Failed: " +
                e.getMessage()
        );
    } finally {

        if (con != null) {
            try {
                con.setAutoCommit(true);
                con.close();
            } catch (SQLException ignored) {
            }
        }
    }

} catch (NumberFormatException e) {

    JOptionPane.showMessageDialog(
        this,
        "User ID and Vehicle ID must be numbers."
    );

} catch (DateTimeParseException e) {

    JOptionPane.showMessageDialog(
        this,
        "Enter date/time in the correct format."
    );
}

}

private void viewReservations() {

    String sql =
        "SELECT r.reservation_id, " +

```

```

        "u.name, " +
        "v.vehicle_number, " +
        "ps.slot_number, " +
        "r.reservation_date, " +
        "r.start_time, " +
        "r.end_time, " +
        "r.status " +
        "FROM reservations r " +
        "JOIN users u ON r.user_id = u.user_id " +
        "JOIN vehicles v ON r.vehicle_id = v.vehicle_id " +
        "JOIN parking_slots ps ON r.slot_id = ps.slot_id " +
        "ORDER BY r.reservation_id";

```

```

try (
    Connection con = DBConnection.getConnection();
    Statement st = con.createStatement();
    ResultSet rs = st.executeQuery(sql)
) {

    outputArea.setText("");

    outputArea.append(
        String.format(
            "%-5s %-15s %-15s %-8s %-12s %-8s %-8s %-12s%n",
            "ID",
            "USER",
            "VEHICLE",
            "SLOT",
            "DATE",
            "START",
            "END",
            "STATUS"
        )
    );

    outputArea.append(
        "-----\n"
    );

    while (rs.next()) {

        outputArea.append(
            String.format(
                "%-5d %-15s %-15s %-8s %-12s %-8s %-8s %-12s%n",
                rs.getInt("reservation_id"),
                rs.getString("name"),

```

```

        rs.getString("vehicle_number"),
        rs.getString("slot_number"),
        rs.getDate("reservation_date"),
        rs.getTime("start_time"),
        rs.getTime("end_time"),
        rs.getString("status")
    )
    );
}

} catch (SQLException e) {

    JOptionPane.showMessageDialog(
        this,
        "Database Error: " +
        e.getMessage()
    );
}
}
}

```

Reservation Details	
User ID:	
Vehicle ID:	
Parking Slot:	2 - S02
Date (YYYY-MM-DD):	2026-08-31
Start Time (HH:MM):	09:00
End Time (HH:MM):	10:00

Reserve Slot

View Reservations

Refresh Slots

Figure 6: Reservation Management

Entry / Exit Management:

```
import javax.swing.*;
import java.awt.*;
import java.sql.*;
import java.time.Duration;
import java.time.LocalDateTime;

public class EntryExit extends JFrame {

    JTextField vehicleIdField;
    JComboBox<String> slotBox;
    JTextField sessionIdField;

    JTextArea outputArea;

    public EntryExit() {

        setTitle("Vehicle Entry and Exit Management");
        setSize(850, 650);
        setLocationRelativeTo(null);
        setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);

        JLabel title = new JLabel(
            "VEHICLE ENTRY / EXIT MANAGEMENT",
            SwingConstants.CENTER
        );

        title.setFont(new Font("Arial", Font.BOLD, 24));

        title.setBorder(
            BorderFactory.createEmptyBorder(20, 10, 20, 10)
        );

        JPanel entryPanel = new JPanel(
            new GridLayout(2, 2, 10, 10)
        );

        entryPanel.setBorder(
            BorderFactory.createTitledBorder(
                "Vehicle Entry"
            )
        );

        vehicleIdField = new JTextField();
```

```
slotBox = new JComboBox<>();

JButton entryButton = new JButton(
    "Record Vehicle Entry"
);

JButton refreshButton = new JButton(
    "Refresh Slots"
);

entryPanel.add(
    new JLabel("Vehicle ID:")
);

entryPanel.add(vehicleIdField);

entryPanel.add(
    new JLabel("Parking Slot:")
);

entryPanel.add(slotBox);

JPanel exitPanel = new JPanel(
    new GridLayout(2, 2, 10, 10)
);

exitPanel.setBorder(
    BorderFactory.createTitledBorder(
        "Vehicle Exit"
    )
);

sessionIdField = new JTextField();

JButton exitButton = new JButton(
    "Record Vehicle Exit"
);

JButton viewButton = new JButton(
    "View Parking History"
);

exitPanel.add(
    new JLabel("Session ID:")
);
```



```
exitPanel.add(sessionIdField);

exitPanel.add(exitButton);
exitPanel.add(viewButton);

outputArea = new JTextArea();

outputArea.setEditable(false);

outputArea.setFont(
    new Font(
        "Monospaced",
        Font.PLAIN,
        14
    )
);

JScrollPane scrollPane =
    new JScrollPane(outputArea);

JPanel topPanel =
    new JPanel(new GridLayout(2, 1, 10, 10));

topPanel.setBorder(
    BorderFactory.createEmptyBorder(
        15, 25, 15, 25
    )
);

topPanel.add(entryPanel);
topPanel.add(exitPanel);

JPanel buttonPanel =
    new JPanel(new GridLayout(1, 2, 15, 15));

buttonPanel.add(entryButton);
buttonPanel.add(refreshButton);

JPanel mainPanel =
    new JPanel(new BorderLayout(10, 10));

mainPanel.setBorder(
    BorderFactory.createEmptyBorder(
        10, 25, 20, 25
    )
);
```

```

);

mainPanel.add(
    topPanel,
    BorderLayout.NORTH
);

mainPanel.add(
    buttonPanel,
    BorderLayout.CENTER
);

mainPanel.add(
    scrollPane,
    BorderLayout.SOUTH
);

add(title, BorderLayout.NORTH);
add(mainPanel, BorderLayout.CENTER);

entryButton.addActionListener(
    e -> recordEntry()
);

refreshButton.addActionListener(
    e -> loadAvailableSlots()
);

exitButton.addActionListener(
    e -> recordExit()
);

viewButton.addActionListener(
    e -> viewHistory()
);

loadAvailableSlots();

setVisible(true);
}

private void loadAvailableSlots() {

    slotBox.removeAllItems();

    String sql =

```

```

        "SELECT slot_id, slot_number " +
        "FROM parking_slots " +
        "WHERE slot_status = 'Available' " +
        "ORDER BY slot_id";

try (
    Connection con =
        DBConnection.getConnection();

    Statement st =
        con.createStatement();

    ResultSet rs =
        st.executeQuery(sql)
) {

    while (rs.next()) {

        slotBox.addItem(
            rs.getInt("slot_id")
            + " - "
            + rs.getString("slot_number")
        );
    }

} catch (SQLException e) {

    JOptionPane.showMessageDialog(
        this,
        "Database Error: "
        + e.getMessage()
    );
}

private void recordEntry() {

    String vehicleText =
        vehicleIdField.getText().trim();

    if (vehicleText.isEmpty()) {

        JOptionPane.showMessageDialog(
            this,
            "Enter Vehicle ID."
        );
    }
}

```

```

        return;
    }

    if (slotBox.getSelectedItem() == null) {

        JOptionPane.showMessageDialog(
            this,
            "No available parking slot."
        );

        return;
    }

    try {

        int vehicleId =
            Integer.parseInt(vehicleText);

        String selectedSlot =
            slotBox.getSelectedItem()
                .toString();

        int slotId =
            Integer.parseInt(
                selectedSlot
                    .split(" - ")[0]
            );

        Connection con =
            DBConnection.getConnection();

        con.setAutoCommit(false);

        try {

            String vehicleCheck =
                "SELECT vehicle_id " +
                "FROM vehicles " +
                "WHERE vehicle_id = ?";

            try (
                PreparedStatement ps =
                    con.prepareStatement(
                        vehicleCheck
                    )
            ) {

```

```

    ) {

        ps.setInt(
            1,
            vehicleId
        );

        ResultSet rs =
            ps.executeQuery();

        if (!rs.next()) {

            con.rollback();

            JOptionPane.showMessageDialog(
                this,
                "Vehicle ID does not exist."
            );

            return;
        }
    }

    String slotCheck =
        "SELECT slot_status " +
        "FROM parking_slots " +
        "WHERE slot_id = ? " +
        "FOR UPDATE";

    try (
        PreparedStatement ps =
            con.prepareStatement(
                slotCheck
            )
    ) {

        ps.setInt(
            1,
            slotId
        );

        ResultSet rs =
            ps.executeQuery();

        if (!rs.next()) {

```

```

        con.rollback();

        JOptionPane.showMessageDialog(
            this,
            "Parking slot not found."
        );

        return;
    }

    if (!rs.getString(
        "slot_status"
    ).equals("Available")) {

        con.rollback();

        JOptionPane.showMessageDialog(
            this,
            "Slot is not available."
        );

        loadAvailableSlots();

        return;
    }
}

String activeCheck =
    "SELECT session_id " +
    "FROM parking_sessions " +
    "WHERE vehicle_id = ? " +
    "AND status = 'Active'";

try (
    PreparedStatement ps =
        con.prepareStatement(
            activeCheck
        )
) {

    ps.setInt(
        1,
        vehicleId
    );

    ResultSet rs =

```

```

        ps.executeQuery();

    if (rs.next()) {

        con.rollback();

        JOptionPane.showMessageDialog(
            this,
            "Vehicle already has an active parking session."
        );

        return;
    }
}

String insertSql =
    "INSERT INTO parking_sessions " +
        "(vehicle_id, slot_id, entry_time, status) " +
        "VALUES (?, ?, ?, 'Active')";

int sessionId = 0;

try (
    PreparedStatement ps =
        con.prepareStatement(
            insertSql,
            Statement.RETURN_GENERATED_KEYS
        )
) {

    ps.setInt(
        1,
        vehicleId
    );

    ps.setInt(
        2,
        slotId
    );

    ps.setTimestamp(
        3,
        Timestamp.valueOf(
            LocalDateTime.now()
        )
    );

};

```

```

        ps.executeUpdate();

        ResultSet keys =
            ps.getGeneratedKeys();

        if (keys.next()) {
            sessionId =
                keys.getInt(1);
        }
    }

    String updateSql =
        "UPDATE parking_slots " +
        "SET slot_status = 'Occupied' " +
        "WHERE slot_id = ?";

    try (
        PreparedStatement ps =
            con.prepareStatement(
                updateSql
            )
    ) {

        ps.setInt(
            1,
            slotId
        );

        ps.executeUpdate();
    }

    con.commit();

    JOptionPane.showMessageDialog(
        this,
        "Vehicle Entry Recorded!\n" +
        "Session ID: " +
        sessionId
    );

    vehicleIdField.setText("");

    loadAvailableSlots();

    viewHistory();

```



```

    } catch (SQLException e) {

        con.rollback();

        JOptionPane.showMessageDialog(
            this,
            "Entry Failed: "
            + e.getMessage()
        );

    } finally {

        con.setAutoCommit(true);
        con.close();
    }

} catch (NumberFormatException e) {

    JOptionPane.showMessageDialog(
        this,
        "Vehicle ID must be a number."
    );

} catch (SQLException e) {

    JOptionPane.showMessageDialog(
        this,
        "Database Error: "
        + e.getMessage()
    );
}
}

private void recordExit() {

    String sessionText =
        sessionIdField.getText().trim();

    if (sessionText.isEmpty()) {

        JOptionPane.showMessageDialog(
            this,
            "Enter Session ID."
        );
    }
}

```

```

    return;
}

try {

    int sessionId =
        Integer.parseInt(
            sessionText
        );

    Connection con =
        DBConnection.getConnection();

    con.setAutoCommit(false);

    try {

        String selectSql =
            "SELECT ps.vehicle_id, " +
            "ps.slot_id, " +
            "ps.entry_time, " +
            "ps.status, " +
            "v.vehicle_type " +
            "FROM parking_sessions ps " +
            "JOIN vehicles v " +
            "ON ps.vehicle_id = v.vehicle_id " +
            "WHERE ps.session_id = ? " +
            "FOR UPDATE";

        int vehicleId;
        int slotId;
        Timestamp entryTimestamp;
        String status;
        String vehicleType;

        try (
            PreparedStatement ps =
                con.prepareStatement(
                    selectSql
                )
        ) {

            ps.setInt(
                1,
                sessionId
            );

```

```

ResultSet rs =
    ps.executeQuery();

if (!rs.next()) {

    con.rollback();

    JOptionPane.showMessageDialog(
        this,
        "Session ID not found."
    );

    return;
}

vehicleId =
    rs.getInt(
        "vehicle_id"
    );

slotId =
    rs.getInt(
        "slot_id"
    );

entryTimestamp =
    rs.getTimestamp(
        "entry_time"
    );

status =
    rs.getString(
        "status"
    );

vehicleType =
    rs.getString(
        "vehicle_type"
    );
}

if (!status.equals("Active")) {

    con.rollback();

```

```
JOptionPane.showMessageDialog(  
    this,  
    "This session is already completed."  
);  
  
return;  
}
```

```
LocalDateTime entryTime =  
    entryTimestamp.toLocalDateTime();
```

```
LocalDateTime exitTime =  
    LocalDateTime.now();
```

```
long minutes =  
    Duration.between(  
        entryTime,  
        exitTime  
    ).toMinutes();
```

```
if (minutes < 1) {  
    minutes = 1;  
}
```

```
double hourlyRate;
```

```
switch (vehicleType) {  
  
    case "Bike":  
        hourlyRate = 10.0;  
        break;  
  
    case "Car":  
        hourlyRate = 20.0;  
        break;  
  
    case "Van":  
        hourlyRate = 30.0;  
        break;  
  
    case "Bus":  
        hourlyRate = 40.0;  
        break;  
  
    default:  
        hourlyRate = 15.0;
```

```

    }

    double fee =
        Math.ceil(
            minutes / 60.0
        ) * hourlyRate;

    String updateSession =
        "UPDATE parking_sessions " +
        "SET exit_time = ?, " +
        "duration_minutes = ?, " +
        "parking_fee = ?, " +
        "status = 'Completed' " +
        "WHERE session_id = ?";

    try (
        PreparedStatement ps =
            con.prepareStatement(
                updateSession
            )
    ) {

        ps.setTimestamp(
            1,
            Timestamp.valueOf(
                exitTime
            )
        );

        ps.setLong(
            2,
            minutes
        );

        ps.setDouble(
            3,
            fee
        );

        ps.setInt(
            4,
            sessionId
        );

        ps.executeUpdate();
    }

```

```

String updateSlot =
    "UPDATE parking_slots " +
        "SET slot_status = 'Available' " +
        "WHERE slot_id = ?";

try (
    PreparedStatement ps =
        con.prepareStatement(
            updateSlot
        )
) {

    ps.setInt(
        1,
        slotId
    );

    ps.executeUpdate();
}

con.commit();

JOptionPane.showMessageDialog(
    this,
    "Vehicle Exit Recorded!\n\n" +
        "Vehicle Type: "
        + vehicleType +
        "\nDuration: "
        + minutes +
        " minutes" +
        "\nParking Fee: ₹"
        + String.format(
            "%.2f",
            fee
        )
);

sessionIdField.setText("");

loadAvailableSlots();

viewHistory();
} catch (SQLException e) {

```

```

        con.rollback();

        JOptionPane.showMessageDialog(
            this,
            "Exit Failed: "
                + e.getMessage()
        );
    } finally {

        con.setAutoCommit(true);
        con.close();
    }
} catch (NumberFormatException e) {

    JOptionPane.showMessageDialog(
        this,
        "Session ID must be a number."
    );
} catch (SQLException e) {

    JOptionPane.showMessageDialog(
        this,
        "Database Error: "
            + e.getMessage()
    );
}
}

private void viewHistory() {

    String sql =
        "SELECT ps.session_id, " +
        "v.vehicle_number, " +
        "v.vehicle_type, " +
        "s.slot_number, " +
        "ps.entry_time, " +
        "ps.exit_time, " +
        "ps.duration_minutes, " +
        "ps.parking_fee, " +
        "ps.status " +
        "FROM parking_sessions ps " +
        "JOIN vehicles v " +
        "ON ps.vehicle_id = v.vehicle_id " +

```

```

        "JOIN parking_slots s " +
        "ON ps.slot_id = s.slot_id " +
        "ORDER BY ps.session_id";

try (
    Connection con =
        DBConnection.getConnection();

    Statement st =
        con.createStatement();

    ResultSet rs =
        st.executeQuery(sql)
) {

    outputArea.setText("");

    outputArea.append(
        String.format(
            "%-5s %-15s %-10s %-7s %-20s %-20s %-8s %-10s %-10s%n",
            "ID",
            "VEHICLE",
            "TYPE",
            "SLOT",
            "ENTRY",
            "EXIT",
            "MIN",
            "FEE",
            "STATUS"
        )
    );

    outputArea.append(
        "-----\n"
    );

    while (rs.next()) {

        outputArea.append(
            String.format(
                "%-5d %-15s %-10s %-7s %-20s %-20s %-8s %-10.2f %-10s%n",
                rs.getInt(
                    "session_id"
                ),
                rs.getString(

```



```

        "vehicle_number"
    ),
    rs.getString(
        "vehicle_type"
    ),
    rs.getString(
        "slot_number"
    ),
    rs.getTimestamp(
        "entry_time"
    ),
    rs.getTimestamp(
        "exit_time"
    ),
    rs.getObject(
        "duration_minutes"
    ),
    rs.getDouble(
        "parking_fee"
    ),
    rs.getString(
        "status"
    )
    )
    );
}

} catch (SQLException e) {

    JOptionPane.showMessageDialog(
        this,
        "Database Error: "
        + e.getMessage()
    );
}
}
}

```

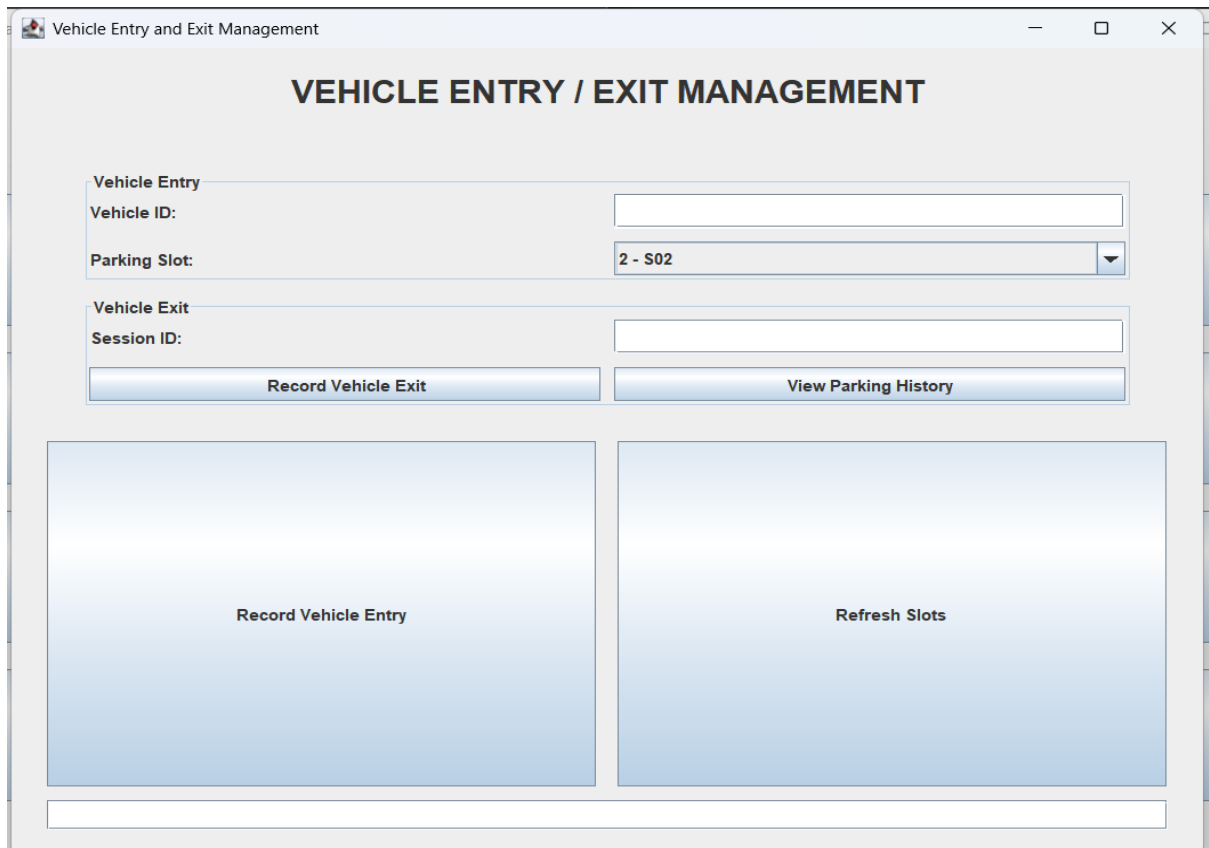


Figure 7: Entry / Exit Management

Payment Management

```
import javax.swing.*;
import java.awt.*;
import java.sql.*;

public class Payments extends JFrame {

    JTextField sessionIdField;
    JTextField amountField;

    JComboBox<String> methodBox;
    JComboBox<String> statusBox;

    JTextArea paymentArea;

    public Payments() {

        setTitle("Parking Payment Management");
        setSize(1050, 700);
```

```
setLocationRelativeTo(null);
setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);
setLayout(new BorderLayout());
```

```
JLabel title = new JLabel(
    "PARKING PAYMENT MANAGEMENT",
    SwingConstants.CENTER
);
```

```
title.setFont(
    new Font("Arial", Font.BOLD, 28)
);
```

```
title.setBorder(
    BorderFactory.createEmptyBorder(
        25, 10, 20, 10
    )
);
```

```
add(title, BorderLayout.NORTH);
```

```
JPanel detailsPanel = new JPanel(
    new GridLayout(3, 2, 15, 15)
);
```

```
detailsPanel.setBorder(
    BorderFactory.createTitledBorder(
        "Payment Details"
    )
);
```

```
JLabel sessionLabel =
    new JLabel("Session ID:");
```

```
JLabel amountLabel =
    new JLabel("Amount:");
```

```
JLabel methodLabel =
    new JLabel("Payment Method:");
```

```
sessionIdField =
    new JTextField();
```

```
amountField =
    new JTextField();
```

```

methodBox =
    new JComboBox<>(  

        new String[]{  

            "Cash",  

            "UPI",  

            "Card"  

        }  

    );

detailsPanel.add(sessionLabel);
detailsPanel.add(sessionIdField);

detailsPanel.add(amountLabel);
detailsPanel.add(amountField);

detailsPanel.add(methodLabel);
detailsPanel.add(methodBox);

JPanel centerPanel =
    new JPanel(new BorderLayout());

centerPanel.setBorder(  

    BorderFactory.createEmptyBorder(  

        10, 30, 10, 30  

    )  

);

centerPanel.add(  

    detailsPanel,  

    BorderLayout.NORTH  

);

paymentArea =
    new JTextArea();

paymentArea.setEditable(false);

paymentArea.setFont(  

    new Font(  

        "Monospaced",  

        Font.PLAIN,  

        13  

    )  

);

JScrollPane scrollPane =

```

```
        new JScrollPane(paymentArea);

scrollPane.setBorder(
    BorderFactory.createTitledBorder(
        "Payment Records"
    )
);

centerPanel.add(
    scrollPane,
    BorderLayout.CENTER
);

add(
    centerPanel,
    BorderLayout.CENTER
);

JButton addButton =
    new JButton("Add Payment");

JButton viewButton =
    new JButton("View Payments");

JButton refreshButton =
    new JButton("Refresh");

JButton clearButton =
    new JButton("Clear");

JButton closeButton =
    new JButton("Close");

Font buttonFont =
    new Font(
        "Arial",
        Font.BOLD,
        15
    );

addButton.setFont(buttonFont);
viewButton.setFont(buttonFont);
refreshButton.setFont(buttonFont);
clearButton.setFont(buttonFont);
closeButton.setFont(buttonFont);
```

```

JPanel buttonPanel =
    new JPanel(
        new GridLayout(
            1,
            5,
            15,
            10
        )
    );

buttonPanel.setBorder(
    BorderFactory.createEmptyBorder(
        15, 30, 25, 30
    )
);

buttonPanel.add(addButton);
buttonPanel.add(viewButton);
buttonPanel.add(refreshButton);
buttonPanel.add(clearButton);
buttonPanel.add(closeButton);

add(
    buttonPanel,
    BorderLayout.SOUTH
);

addButton.addActionListener(
    e -> addPayment()
);

viewButton.addActionListener(
    e -> viewPayments()
);

refreshButton.addActionListener(
    e -> viewPayments()
);

clearButton.addActionListener(
    e -> clearFields()
);

closeButton.addActionListener(
    e -> dispose()
);

```

```

        viewPayments();

        setVisible(true);
    }

    private void addPayment() {

        String sessionText =
            sessionIdField.getText().trim();

        String amountText =
            amountField.getText().trim();

        if (sessionText.isEmpty()) {

            JOptionPane.showMessageDialog(
                this,
                "Please enter Session ID."
            );

            return;
        }

        if (amountText.isEmpty()) {

            JOptionPane.showMessageDialog(
                this,
                "Please enter Amount."
            );

            return;
        }

        int sessionId;

        double amount;

        try {

            sessionId =
                Integer.parseInt(sessionText);

        } catch (NumberFormatException e) {

            JOptionPane.showMessageDialog(

```

```

        this,
        "Session ID must be a number."
    );

    return;
}

try {

    amount =
        Double.parseDouble(amountText);

} catch (NumberFormatException e) {

    JOptionPane.showMessageDialog(
        this,
        "Amount must be a valid number."
    );

    return;
}

if (amount <= 0) {

    JOptionPane.showMessageDialog(
        this,
        "Amount must be greater than zero."
    );

    return;
}

String method =
    methodBox.getSelectedItem().toString();

String sql =
    "INSERT INTO payments " +
        "(session_id, amount, payment_method) " +
        "VALUES (?, ?, ?)";

try (
    Connection con =
        DBConnection.getConnection();

    PreparedStatement ps =
        con.prepareStatement(sql)

```



```

    ) {

        ps.setInt(
            1,
            sessionId
        );

        ps.setDouble(
            2,
            amount
        );

        ps.setString(
            3,
            method
        );

        ps.executeUpdate();

        JOptionPane.showMessageDialog(
            this,
            "Payment Added Successfully!"
        );

        clearFields();

        viewPayments();

    } catch (SQLException e) {

        JOptionPane.showMessageDialog(
            this,
            "Database Error:\n"
                + e.getMessage()
        );
    }
}

private void viewPayments() {

    String sql =
        "SELECT * FROM payments " +
        "ORDER BY payment_id";

    try (
        Connection con =

```

```

        DBConnection.getConnection();

        PreparedStatement ps =
            con.prepareStatement(sql);

        ResultSet rs =
            ps.executeQuery()
    ) {

        paymentArea.setText("");

        paymentArea.append(
            "PARKING PAYMENT RECORDS\n"
        );

        paymentArea.append(

"=====
====\n"
        );

        paymentArea.append(
            String.format(
                "%-12s %-12s %-12s %-18s %-22s %-15s\n",
                "Payment ID",
                "Session ID",
                "Amount",
                "Payment Method",
                "Payment Date",
                "Status"
            )
        );

        paymentArea.append(
            "-----\n"
        );

        while (rs.next()) {

            int paymentId =
                rs.getInt("payment_id");

            int sessionId =
                rs.getInt("session_id");

            double amount =

```

```

        rs.getDouble("amount");

String method =
    rs.getString(
        "payment_method"
    );

String date =
    rs.getString(
        "payment_date"
    );

String status =
    rs.getString(
        "payment_status"
    );

paymentArea.append(
    String.format(
        "%-12d %-12d ₹%-11.2f %-18s %-22s %-15s%n",
        paymentId,
        sessionId,
        amount,
        method,
        date,
        status
    )
);
}

} catch (SQLException e) {

    paymentArea.setText("");

    JOptionPane.showMessageDialog(
        this,
        "Database Error:\n"
        + e.getMessage()
    );
}

}

private void clearFields() {

    sessionIdField.setText("");

```

```

        amountField.setText("");

        methodBox.setSelectedIndex(0);
    }

    public static void main(String[] args) {

        SwingUtilities.invokeLater(
            () -> new Payments()
        );
    }
}

```

PARKING PAYMENT MANAGEMENT

Payment Details

Session ID:

Amount:

Payment Method:

Payment Records

PARKING PAYMENT RECORDS

Payment ID	Session ID	Amount	Payment Method	Payment Date	Status
1	1	₹10.00	UPI	2026-08-31 22:18:05	Paid
2	1	₹10.00	Cash	2026-08-31 22:36:47	Paid

Buttons: Add Payment, View Payments, Refresh, Clear, Close

Figure 8: Payment Management

Violation Management:

```

import javax.swing.*;
import java.awt.*;
import java.sql.*;

public class Violations extends JFrame {

    JTextField userIdField;
    JTextField vehicleIdField;

```

```

TextField typeField;
TextField descriptionField;
TextField fineField;
JComboBox<String> statusBox;
TextArea outputArea;

public Violations() {

    setTitle("Parking Violation Management");
    setSize(1000, 700);
    setLocationRelativeTo(null);
    setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);

    JLabel title = new JLabel(
        "PARKING VIOLATION MANAGEMENT",
        SwingConstants.CENTER
    );

    title.setFont(new Font("Arial", Font.BOLD, 28));
    title.setBorder(
        BorderFactory.createEmptyBorder(25, 10, 25, 10)
    );

    JPanel formPanel = new JPanel(
        new GridLayout(6, 2, 10, 12)
    );

    formPanel.setBorder(
        BorderFactory.createTitledBorder(
            "Violation Details"
        )
    );

    userIdField = new TextField();
    vehicleIdField = new TextField();
    typeField = new TextField();
    descriptionField = new TextField();
    fineField = new TextField();

    statusBox = new JComboBox<>(
        new String[]{
            "Pending",
            "Paid"
        }
    );

```

```
formPanel.add(new JLabel("User ID:"));
formPanel.add(userIdField);

formPanel.add(new JLabel("Vehicle ID:"));
formPanel.add(vehicleIdField);

formPanel.add(new JLabel("Violation Type:"));
formPanel.add(typeField);

formPanel.add(new JLabel("Description:"));
formPanel.add(descriptionField);

formPanel.add(new JLabel("Fine Amount:"));
formPanel.add(fineField);

formPanel.add(new JLabel("Status:"));
formPanel.add(statusBox);

JButton addButton =
    new JButton("Add Violation");

JButton viewButton =
    new JButton("View Violations");

JButton refreshButton =
    new JButton("Refresh");

JPanel buttonPanel = new JPanel(
    new GridLayout(1, 3, 15, 15)
);

buttonPanel.add(addButton);
buttonPanel.add(viewButton);
buttonPanel.add(refreshButton);

outputArea = new JTextArea();

outputArea.setEditable(false);

outputArea.setFont(
    new Font("Monospaced", Font.PLAIN, 13)
);

JScrollPane scrollPane =
    new JScrollPane(outputArea);
```

```

JPanel mainPanel = new JPanel(
    new BorderLayout(15, 15)
);

mainPanel.setBorder(
    BorderFactory.createEmptyBorder(
        20, 35, 20, 35
    )
);

mainPanel.add(
    formPanel,
    BorderLayout.NORTH
);

mainPanel.add(
    buttonPanel,
    BorderLayout.CENTER
);

mainPanel.add(
    scrollPane,
    BorderLayout.SOUTH
);

add(title, BorderLayout.NORTH);
add(mainPanel, BorderLayout.CENTER);

addButton.addActionListener(
    e -> addViolation()
);

viewButton.addActionListener(
    e -> viewViolations()
);

refreshButton.addActionListener(
    e -> clearFields()
);

setVisible(true);
}

private void addViolation() {

    String userText =

```

```

        userIdField.getText().trim();

String vehicleText =
    vehicleIdField.getText().trim();

String violationType =
    typeField.getText().trim();

String description =
    descriptionField.getText().trim();

String fineText =
    fineField.getText().trim();

String status =
    statusBox.getSelectedItem().toString();

if (
        userIdText.isEmpty()
            ||
            vehicleText.isEmpty()
            ||
            violationType.isEmpty()
            ||
            fineText.isEmpty()
) {

    JOptionPane.showMessageDialog(
        this,
        "Please fill User ID, Vehicle ID, Violation Type and Fine."
    );

    return;
}

try {

    int userId =
        Integer.parseInt(userIdText);

    int vehicleId =
        Integer.parseInt(vehicleText);

    double fine =
        Double.parseDouble(fineText);

```



```

if (fine < 0) {

    JOptionPane.showMessageDialog(
        this,
        "Fine amount cannot be negative."
    );

    return;
}

String sql =
    "INSERT INTO violations " +
        "(user_id, vehicle_id, violation_type, " +
        "description, fine_amount, status) " +
        "VALUES (?, ?, ?, ?, ?, ?)";

try (
    Connection con =
        DBConnection.getConnection();

    PreparedStatement ps =
        con.prepareStatement(
            sql,
            Statement.RETURN_GENERATED_KEYS
        )
) {

    ps.setInt(1, userId);
    ps.setInt(2, vehicleId);
    ps.setString(3, violationType);
    ps.setString(4, description);
    ps.setDouble(5, fine);
    ps.setString(6, status);

    ps.executeUpdate();

    int violationId = 0;

    ResultSet keys =
        ps.getGeneratedKeys();

    if (keys.next()) {
        violationId =
            keys.getInt(1);
    }
}

```

```

        JOptionPane.showMessageDialog(
            this,
            "Violation Added Successfully!\n\n"
                + "Violation ID: "
                + violationId
                + "\nUser ID: "
                + userId
                + "\nVehicle ID: "
                + vehicleId
                + "\nViolation Type: "
                + violationType
                + "\nFine: ₹"
                + String.format(
                    "%.2f",
                    fine
                )
        );
    }

    clearFields();
    viewViolations();

} catch (NumberFormatException e) {

    JOptionPane.showMessageDialog(
        this,
        "User ID, Vehicle ID and Fine must be valid numbers."
    );

} catch (SQLException e) {

    JOptionPane.showMessageDialog(
        this,
        "Database Error:\n"
            + e.getMessage()
    );
}

}

private void viewViolations() {

    String sql =
        "SELECT violation_id, user_id, vehicle_id, " +
        "violation_type, description, fine_amount, " +
        "violation_date, status " +
        "FROM violations " +

```

```

        "ORDER BY violation_id";

try (
    Connection con =
        DBConnection.getConnection();

    PreparedStatement ps =
        con.prepareStatement(sql);

    ResultSet rs =
        ps.executeQuery()
) {

    outputArea.setText("");

    outputArea.append(
        String.format(
            "%-5s %-7s %-9s %-18s %-22s %-10s %-20s %-10s%n",
            "ID",
            "USER",
            "VEHICLE",
            "TYPE",
            "DESCRIPTION",
            "FINE",
            "DATE",
            "STATUS"
        )
    );

    outputArea.append(
        "-----\n"
    );

    while (rs.next()) {

        outputArea.append(
            String.format(
                "%-5d %-7d %-9d %-18s %-22s ₹%-9.2f %-20s %-10s%n",
                rs.getInt("violation_id"),
                rs.getInt("user_id"),
                rs.getInt("vehicle_id"),
                rs.getString("violation_type"),
                rs.getString("description"),
                rs.getDouble("fine_amount"),
                rs.getTimestamp("violation_date"),
            )
        );
    }
}

```

```

        rs.getString("status")
    )
    );
}

} catch (SQLException e) {

    JOptionPane.showMessageDialog(
        this,
        "Database Error:\n"
        + e.getMessage()
    );
}
}

private void clearFields() {

    userIdField.setText("");
    vehicleIdField.setText("");
    typeField.setText("");
    descriptionField.setText("");
    fineField.setText("");

    statusBox.setSelectedIndex(0);
}
}

```

ID	User ID	Vehicle ID	Description	Fine Amount	Date	Status
1	5	5	wrong parking	vehicle parking in wrong slot	₹100.00	2026-08-31 22:27:18.0 Pending

Figure 9: Violation Management

Reports GUI

```
import javax.swing.*;
import java.awt.*;
import java.sql.*;

public class Reports extends JFrame {

    JLabel usersLabel;
    JLabel vehiclesLabel;
    JLabel slotsLabel;
    JLabel availableLabel;
    JLabel occupiedLabel;
    JLabel reservedLabel;
    JLabel reservationsLabel;
    JLabel paymentsLabel;
    JLabel violationsLabel;
    JLabel revenueLabel;
    JLabel finesLabel;

    JTextArea reportArea;

    public Reports() {

        setTitle("Smart Campus Parking - Reports");
        setSize(1100, 750);
        setLocationRelativeTo(null);
        setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);
        setLayout(new BorderLayout(15, 15));

        JLabel title = new JLabel(
            "SMART CAMPUS PARKING - REPORTS",
            SwingConstants.CENTER
        );

        title.setFont(new Font("Arial", Font.BOLD, 28));

        title.setBorder(
            BorderFactory.createEmptyBorder(
                20, 10, 10, 10
            )
        );

        add(title, BorderLayout.NORTH);
```

```

JPanel summaryPanel = new JPanel(
    new GridLayout(6, 2, 15, 12)
);

summaryPanel.setBorder(
    BorderFactory.createTitledBorder(
        "Parking System Summary"
    )
);

usersLabel = new JLabel("Total Users: 0");
vehiclesLabel = new JLabel("Total Vehicles: 0");
slotsLabel = new JLabel("Total Slots: 0");
availableLabel = new JLabel("Available Slots: 0");
occupiedLabel = new JLabel("Occupied Slots: 0");
reservedLabel = new JLabel("Reserved Slots: 0");
reservationsLabel = new JLabel("Total Reservations: 0");
paymentsLabel = new JLabel("Total Payments: 0");
violationsLabel = new JLabel("Total Violations: 0");
revenueLabel = new JLabel("Payment Revenue: ₹0.00");
finesLabel = new JLabel("Total Fines: ₹0.00");

Font font = new Font(
    "Arial",
    Font.BOLD,
    15
);

usersLabel.setFont(font);
vehiclesLabel.setFont(font);
slotsLabel.setFont(font);
availableLabel.setFont(font);
occupiedLabel.setFont(font);
reservedLabel.setFont(font);
reservationsLabel.setFont(font);
paymentsLabel.setFont(font);
violationsLabel.setFont(font);
revenueLabel.setFont(font);
finesLabel.setFont(font);

summaryPanel.add(usersLabel);
summaryPanel.add(vehiclesLabel);

summaryPanel.add(slotsLabel);
summaryPanel.add(availableLabel);

```

```

summaryPanel.add(occupiedLabel);
summaryPanel.add(reservedLabel);

summaryPanel.add(reservationsLabel);
summaryPanel.add(paymentsLabel);

summaryPanel.add(violationsLabel);
summaryPanel.add(revenueLabel);

summaryPanel.add(finesLabel);

JPanel topPanel =
    new JPanel(new BorderLayout());

topPanel.setBorder(
    BorderFactory.createEmptyBorder(
        10, 25, 10, 25
    )
);

topPanel.add(
    summaryPanel,
    BorderLayout.CENTER
);

add(
    topPanel,
    BorderLayout.CENTER
);

reportArea = new JTextArea();

reportArea.setEditable(false);

reportArea.setFont(
    new Font(
        "Monospaced",
        Font.PLAIN,
        13
    )
);

JScrollPane scrollPane =
    new JScrollPane(reportArea);

scrollPane.setBorder(

```

```
        BorderFactory.createTitledBorder(
            "Detailed Report"
        )
    );

    JPanel centerPanel =
        new JPanel(new BorderLayout());

    centerPanel.add(
        topPanel,
        BorderLayout.NORTH
    );

    centerPanel.add(
        scrollPane,
        BorderLayout.CENTER
    );

    add(
        centerPanel,
        BorderLayout.CENTER
    );

    JButton refreshButton =
        new JButton("Refresh Report");

    JButton usersButton =
        new JButton("User Report");

    JButton vehiclesButton =
        new JButton("Vehicle Report");

    JButton paymentsButton =
        new JButton("Payment Report");

    JButton violationsButton =
        new JButton("Violation Report");

    JButton closeButton =
        new JButton("Close");

    JPanel buttonPanel =
        new JPanel(
            new GridLayout(
                1,
                6,
```



```

        10,
        10
    )
);

buttonPanel.setBorder(
    BorderFactory.createEmptyBorder(
        10,
        25,
        20,
        25
    )
);

buttonPanel.add(refreshButton);
buttonPanel.add(usersButton);
buttonPanel.add(vehiclesButton);
buttonPanel.add(paymentsButton);
buttonPanel.add(violationsButton);
buttonPanel.add(closeButton);

add(
    buttonPanel,
    BorderLayout.SOUTH
);

refreshButton.addActionListener(
    e -> loadSummary()
);

usersButton.addActionListener(
    e -> showUsers()
);

vehiclesButton.addActionListener(
    e -> showVehicles()
);

paymentsButton.addActionListener(
    e -> showPayments()
);

violationsButton.addActionListener(
    e -> showViolations()
);

```

```

closeButton.addActionListener(
    e -> dispose()
);

loadSummary();

setVisible(true);
}

private void loadSummary() {

    try (
        Connection con =
            DBConnection.getConnection()
    ) {

        int users =
            getCount(
                con,
                "SELECT COUNT(*) FROM users"
            );

        int vehicles =
            getCount(
                con,
                "SELECT COUNT(*) FROM vehicles"
            );

        int slots =
            getCount(
                con,
                "SELECT COUNT(*) FROM parking_slots"
            );

        int available =
            getCount(
                con,
                "SELECT COUNT(*) FROM parking_slots " +
                    "WHERE slot_status = 'Available'"
            );

        int occupied =
            getCount(
                con,
                "SELECT COUNT(*) FROM parking_slots " +
                    "WHERE slot_status = 'Occupied'"
            );
    }
}

```

```

    );

    int reserved =
        getCount(
            con,
            "SELECT COUNT(*) FROM parking_slots " +
            "WHERE slot_status = 'Reserved'"
        );

    int reservations =
        getCount(
            con,
            "SELECT COUNT(*) FROM reservations"
        );

    int payments =
        getCount(
            con,
            "SELECT COUNT(*) FROM payments"
        );

    int violations =
        getCount(
            con,
            "SELECT COUNT(*) FROM violations"
        );

    double revenue =
        getDouble(
            con,
            "SELECT COALESCE(SUM(amount),0) " +
            "FROM payments " +
            "WHERE payment_status = 'Paid'"
        );

    double fines =
        getDouble(
            con,
            "SELECT COALESCE(SUM(fine_amount),0) " +
            "FROM violations"
        );

    usersLabel.setText(
        "Total Users: " + users
    );

```

```
vehiclesLabel.setText(
    "Total Vehicles: " + vehicles
);

slotsLabel.setText(
    "Total Slots: " + slots
);

availableLabel.setText(
    "Available Slots: " + available
);

occupiedLabel.setText(
    "Occupied Slots: " + occupied
);

reservedLabel.setText(
    "Reserved Slots: " + reserved
);

reservationsLabel.setText(
    "Total Reservations: " + reservations
);

paymentsLabel.setText(
    "Total Payments: " + payments
);

violationsLabel.setText(
    "Total Violations: " + violations
);

revenueLabel.setText(
    "Payment Revenue: ₹"
    + String.format(
        "%.2f",
        revenue
    )
);

finesLabel.setText(
    "Total Fines: ₹"
    + String.format(
        "%.2f",
        fines
    )
);
```

```

);

reportArea.setText("");

reportArea.append(
    "SMART CAMPUS PARKING SYSTEM - SUMMARY\n"
);

reportArea.append(
    "=====\\n\\n"
);

reportArea.append(
    "Total Users      : "
    + users + "\\n"
);

reportArea.append(
    "Total Vehicles    : "
    + vehicles + "\\n"
);

reportArea.append(
    "Total Parking Slots : "
    + slots + "\\n"
);

reportArea.append(
    "Available Slots    : "
    + available + "\\n"
);

reportArea.append(
    "Occupied Slots      : "
    + occupied + "\\n"
);

reportArea.append(
    "Reserved Slots      : "
    + reserved + "\\n"
);

reportArea.append(
    "Total Reservations  : "
    + reservations + "\\n"

```

```

    );

    reportArea.append(
        "Total Payments      : "
        + payments + "\n"
    );

    reportArea.append(
        "Total Violations     : "
        + violations + "\n"
    );

    reportArea.append(
        "Payment Revenue       : ₹"
        + String.format(
            "%.2f",
            revenue
        ) + "\n"
    );

    reportArea.append(
        "Total Fine Amount      : ₹"
        + String.format(
            "%.2f",
            fines
        ) + "\n"
    );

    reportArea.append(
        "\n===== \n"
    );

    reportArea.append(
        "Report generated successfully.\n"
    );

} catch (SQLException e) {

    JOptionPane.showMessageDialog(
        this,
        "Database Error:\n"
        + e.getMessage()
    );
}
}

```

```

private int getCount(
    Connection con,
    String sql
) throws SQLException {

    try (
        PreparedStatement ps =
            con.prepareStatement(sql);

        ResultSet rs =
            ps.executeQuery()
    ) {

        if (rs.next()) {
            return rs.getInt(1);
        }
    }

    return 0;
}

```

```

private double getDouble(
    Connection con,
    String sql
) throws SQLException {

    try (
        PreparedStatement ps =
            con.prepareStatement(sql);

        ResultSet rs =
            ps.executeQuery()
    ) {

        if (rs.next()) {
            return rs.getDouble(1);
        }
    }

    return 0.0;
}

```

```

private void showUsers() {

    String sql =

```

```

        "SELECT user_id, name, email, phone, user_type " +
        "FROM users ORDER BY user_id";

try (
    Connection con =
        DBConnection.getConnection();

    PreparedStatement ps =
        con.prepareStatement(sql);

    ResultSet rs =
        ps.executeQuery()
) {

    reportArea.setText("");

    reportArea.append(
        "USER REPORT\n"
    );

    reportArea.append(
        "=====
        ===\n"
    );

    reportArea.append(
        String.format(
            "%-8s %-20s %-30s %-15s %-12s%n",
            "ID",
            "NAME",
            "EMAIL",
            "PHONE",
            "TYPE"
        )
    );

    reportArea.append(
        "-----\n"
    );

    while (rs.next()) {

        reportArea.append(
            String.format(
                "%-8d %-20s %-30s %-15s %-12s%n",

```



```

        rs.getInt("user_id"),
        rs.getString("name"),
        rs.getString("email"),
        rs.getString("phone"),
        rs.getString("user_type")
    )
    );
}

} catch (SQLException e) {

    JOptionPane.showMessageDialog(
        this,
        "Database Error:\n"
        + e.getMessage()
    );
}
}

private void showVehicles() {

    String sql =
        "SELECT vehicle_id, user_id, vehicle_number, vehicle_type " +
        "FROM vehicles ORDER BY vehicle_id";

    try (
        Connection con =
            DBConnection.getConnection();

        PreparedStatement ps =
            con.prepareStatement(sql);

        ResultSet rs =
            ps.executeQuery()
    ) {

        reportArea.setText("");

        reportArea.append(
            "VEHICLE REPORT\n"
        );

        reportArea.append(
            "===== \n"
        );
    }
}

```

```

reportArea.append(
    String.format(
        "%-8s %-8s %-20s %-15s%n",
        "ID",
        "USER",
        "VEHICLE NUMBER",
        "TYPE"
    )
);

reportArea.append(
    "-----\n"
);

while (rs.next()) {

    reportArea.append(
        String.format(
            "%-8d %-8d %-20s %-15s%n",
            rs.getInt("vehicle_id"),
            rs.getInt("user_id"),
            rs.getString("vehicle_number"),
            rs.getString("vehicle_type")
        )
    );
}

} catch (SQLException e) {

    JOptionPane.showMessageDialog(
        this,
        "Database Error:\n"
        + e.getMessage()
    );
}

}

private void showPayments() {

    String sql =
        "SELECT * FROM payments " +
        "ORDER BY payment_id";

    try (
        Connection con =

```

```

        DBConnection.getConnection();

        PreparedStatement ps =
            con.prepareStatement(sql);

        ResultSet rs =
            ps.executeQuery()
    ) {

        reportArea.setText("");

        reportArea.append(
            "PAYMENT REPORT\n"
        );

        reportArea.append(

"=====
====\n"
        );

        ResultSetMetaData meta =
            rs.getMetaData();

        int columns =
            meta.getColumnCount();

        while (rs.next()) {

            for (int i = 1; i <= columns; i++) {

                reportArea.append(
                    meta.getColumnName(i)
                    + " : "
                    + rs.getString(i)
                    + " "
                );
            }

            reportArea.append("\n");
        }

    } catch (SQLException e) {

        JOptionPane.showMessageDialog(
            this,

```

```

        "Database Error:\n"
        + e.getMessage()
    );
}
}

```

```

private void showViolations() {

```

```

    String sql =
        "SELECT violation_id, user_id, vehicle_id, " +
        "violation_type, description, fine_amount, " +
        "violation_date, status " +
        "FROM violations " +
        "ORDER BY violation_id";

```

```

    try (
        Connection con =
            DBConnection.getConnection();

```

```

        PreparedStatement ps =
            con.prepareStatement(sql);

```

```

        ResultSet rs =
            ps.executeQuery()
    ) {

```

```

        reportArea.setText("");

```

```

        reportArea.append(
            "VIOLATION REPORT\n"
        );

```

```

        reportArea.append(

```

```

"=====
=====\n"
        );

```

```

        reportArea.append(
            String.format(
                "%-5s %-6s %-8s %-18s %-25s %-10s %-10s\n",
                "ID",
                "USER",
                "VEHICLE",
                "TYPE",
                "DESCRIPTION",

```

```

        "FINE",
        "STATUS"
    )
);

reportArea.append(
    "-----\n"
);

while (rs.next()) {

    reportArea.append(
        String.format(
            "%-5d %-6d %-8d %-18s %-25s ₹%-9.2f %-10s%n",
            rs.getInt("violation_id"),
            rs.getInt("user_id"),
            rs.getInt("vehicle_id"),
            rs.getString("violation_type"),
            rs.getString("description"),
            rs.getDouble("fine_amount"),
            rs.getString("status")
        )
    );
}

} catch (SQLException e) {

    JOptionPane.showMessageDialog(
        this,
        "Database Error:\n"
        + e.getMessage()
    );
}

}

public static void main(String[] args) {

    SwingUtilities.invokeLater(
        () -> new Reports()
    );
}
}

```

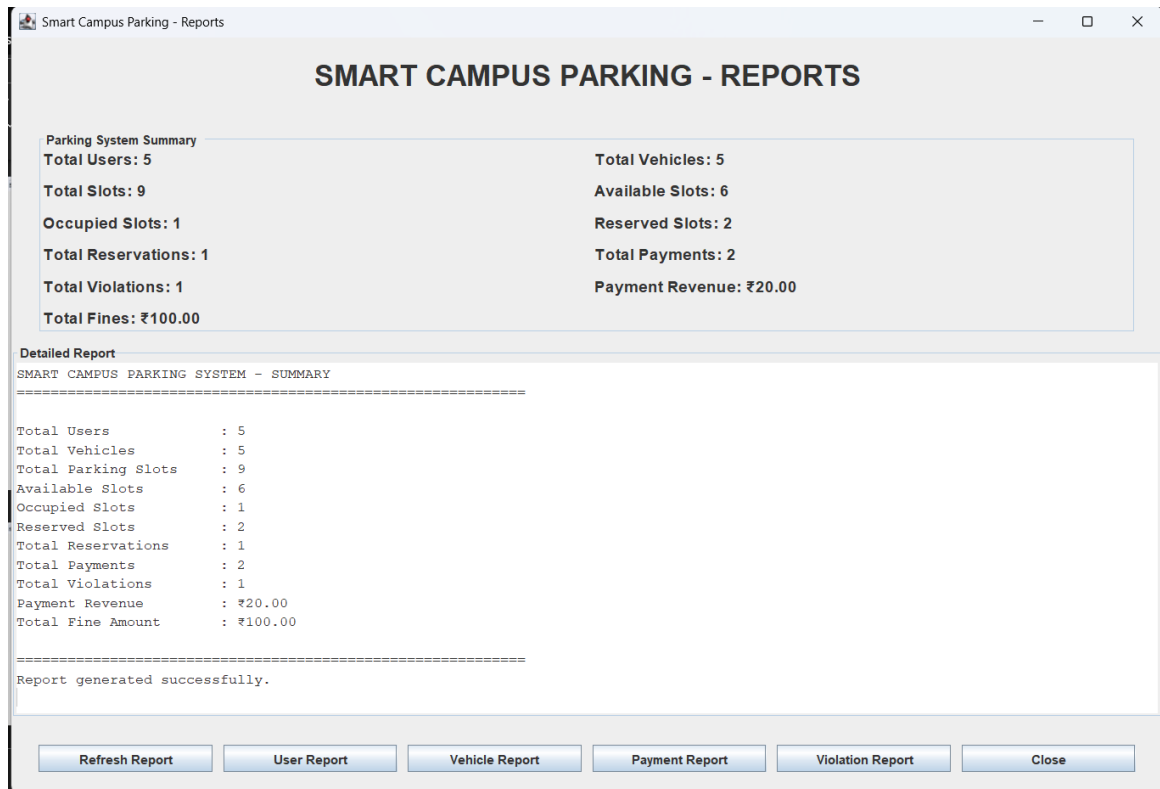


Figure 10: Reports

Result:

The **Smart Campus Parking and Traffic Management System** was successfully implemented using Java Swing, JDBC, and MySQL. The application successfully connects to the database and performs the required parking management operations.